



AutoML#4 -Predicting Algorithm Runtime with TabPFN

Gurmeher Singh Puri, Udit Chawla, Jaydeep Tigaiya
University of Freiburg

Steven Adriaensen

Problem

The Challenge: Predicting Algorithm Performance

- Choosing the right algorithm for a complex task can save significant time and computational resources. However, many state-of-the-art algorithms are randomized, meaning their runtime isn't a single fixed number. One run might be incredibly fast, while the next, on the same problem, could be much slower.
- To truly understand an algorithm's performance, we need to predict its entire **Runtime Distribution (RTD)**. This allows us to see the full picture: from the best-case and worst-case runtimes to the most likely outcomes.

Our Research

This work investigates whether a new, general-purpose model can effectively solve this specialized prediction problem. We compare two powerful approaches:

- The Specialist (DistNet):** A deep learning model from Eggensperger et al. (2018) that was specifically designed to predict algorithm runtime distributions.
- The Challenger (TabPFN v2):** A powerful, pre-trained model designed for a wide variety of tabular data problems..

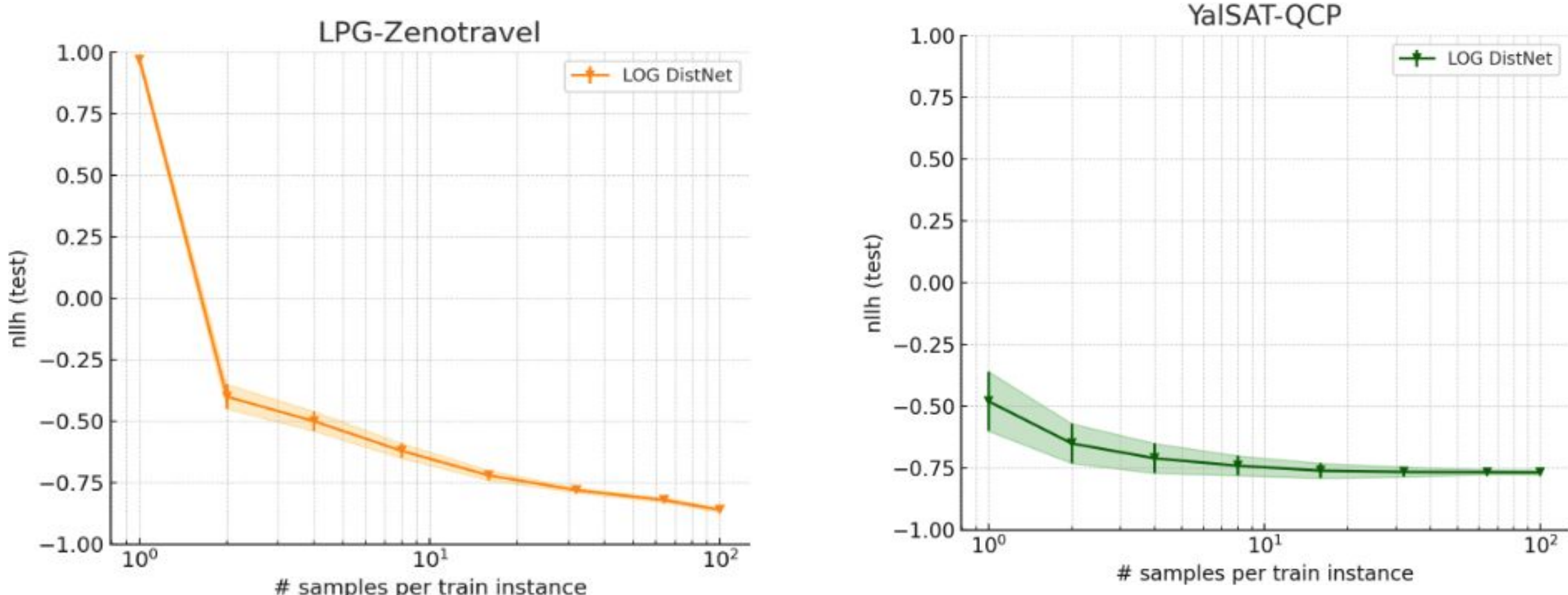
Goal: Evaluate how well TabPFN v2 can model the uncertainty and variability of algorithm runtimes, given a set of problem-specific features, and to see how its predictions stack up against the specialized DistNet approach.

Distnet Baseline Reproduction

- We began by replicating the benchmark performance of the original **DistNet** model.
- Our results successfully align with the Negative Log-Likelihood scores reported for the main network in **Table 3** of Eggensperger et al. (2017).

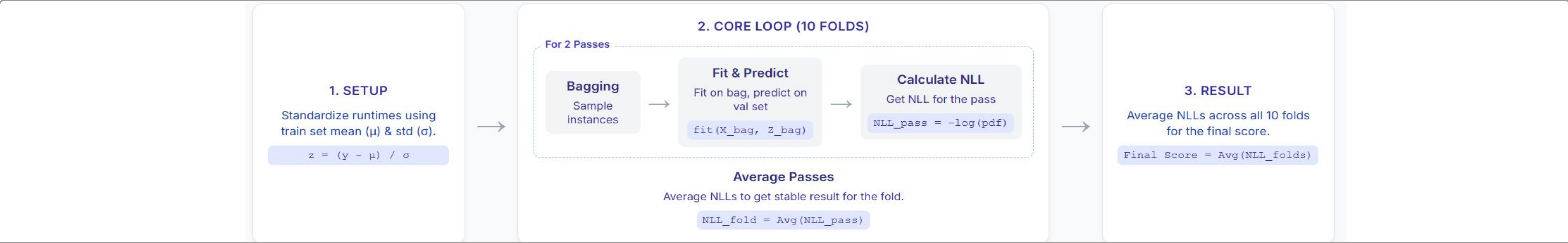
Scenario	Split	Results from Paper (NLLH in LOG Space)		Our Reproduced Results (NLLH in LOG Space)	
		Fitted	DistNet	Fitted	DistNet
Clasp-factoring	train	-0.35	-0.24	-0.351	-0.264
	test	-0.35	-0.16	-0.351	-0.084
Saps-CVVAR	train	-0.88	-0.54	-0.884	-0.673
	test	-0.88	-0.52	-0.884	-0.606
Spear-QCP	train	-1.20	-1.11	-1.201	-1.129
	test	-1.20	-1.10	-1.201	-1.106
YalSAT-QCP	train	-0.78	-0.76	-0.781	-0.767
	test	-0.78	-0.75	-0.781	-0.764
Spear-SWGCP	train	-0.93	-0.48	-0.929	-0.528
	test	-0.93	-0.47	-0.929	-0.455
YalSAT-SWGCP	train	-0.94	-0.81	-0.936	-0.824
	test	-0.94	-0.81	-0.936	-0.800
LPG-Zenotravel	train	-0.90	-0.85	-0.903	-0.860
	test	-0.90	-0.85	-0.903	-0.855

Table 1: Comparison of NLL between the original paper's results and our reproduced results. All values in this table are calculated in the LOG space.



These graphs show the model's **Negative Log-Likelihood (NLLH)** as we provide more runtime samples per training instance. The key takeaway is that DistNet's NLL improves significantly (gets lower) with more data, proving it learns from the runtime distributions.

Pipeline



Limitation for TabPFN

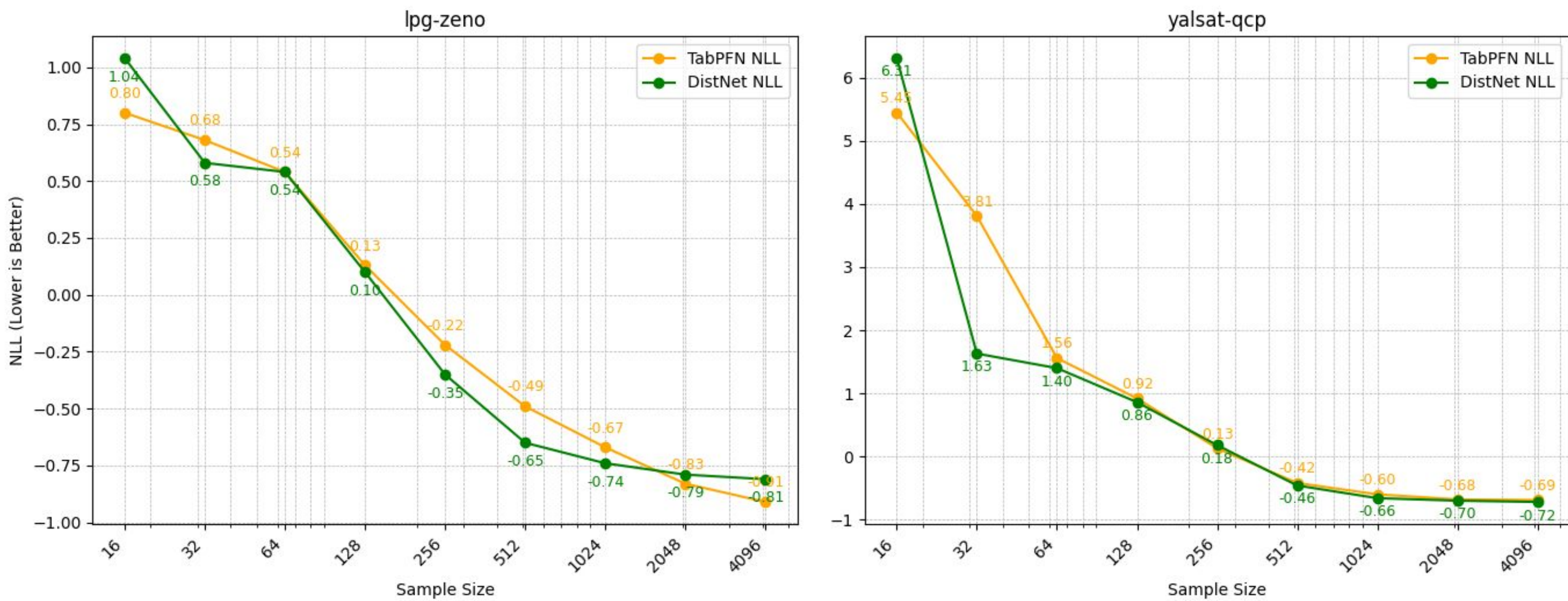
- GPU Memory Limitation:** TabPFN's architecture requires the entire training set to be loaded into GPU memory at once. Our experiments showed a hard limit, causing "out of memory" errors when the training context exceeded approximately **4096 rows**. This is a significant constraint, as the full, flattened runtime datasets are massive, ranging from **200,000 to over 1 million rows**. Consequently, TabPFN could only ever see a tiny fraction of the available data.
- Our Solution (Bagging Pipeline):** To work around this memory constraint, we designed a **bagging pipeline**. Instead of training on the full dataset, we repeatedly trained TabPFN on small, random sub-samples (bags) of 4096 rows and averaged the results. While this allowed us to evaluate the model, it means TabPFN could never leverage all the available training data at once, which likely limited its full learning potential.

Results

- All NLLH scores were calculated in a **Standardized (Z-score) Space**. To achieve this, raw runtimes were transformed using the mean and standard deviation of the training data for each fold.
- This ensures a stable and direct comparison across all scenarios.

Scenario	Mean NLL (in Standardized Z-score Space)	
	TabPFN	DistNet
Clasp-factoring	-1.33	-0.18
LPG-Zeno	-0.91	-0.81
Saps-CVVAR	-0.69	-0.63
Spear-QCP	-1.62	-1.08
Spear-SWGCP	-0.41	-0.45
YalSAT-QCP	-0.69	-0.72
YalSAT-SWGCP	-0.82	-0.79

Table 2: Final NLL comparison. Lower is better. All values are reported in the Standardized (Z-score) Space. For this comparison, DistNet was configured to predict a Normal (Gaussian) distribution. All models were evaluated with a sample size of 4096 and 10 K-folds



These graphs compare the performance (**NLLH, where lower is better**) of TabPFN and DistNet as we increase the number of runtime samples used for training. The plots show that both models generally benefit from more data, with their NLL scores improving as the sample size grows.

Conclusion

Despite its significant memory constraints, the general-purpose TabPFN model shows competitive and often superior performance against the specialized DistNet for predicting algorithm runtime distributions.

References

[1] K. Eggensperger, M. Lindauer & F. Hutter. *Neural Networks for Predicting Algorithm Runtime Distributions*. Proc. IJCAI 2018, pp. 4748-4754.

[2] N. Hollmann, S. Müller, K. Eggensperger & F. Hutter. *TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second*. arXiv preprint arXiv:2207.01848, 2023.

Future Work

- Scaling TabPFN to Larger Datasets:** The most critical next step is to investigate methods to overcome TabPFN's context size limit. Techniques like knowledge distillation, feature selection, or ensembling
- Better scaling + correct NLL:** Test z-score, min-max, robust and log transforms, adding the Jacobian so NLL is still in seconds.
- Optimal Data Transformation:** We can revisit the log transform, which is theoretically better for heavy-tailed runtime data. This involves carefully implementing the correct Jacobian correction
- Data Augmentation:** We can explore using data augmentation techniques to improve model robustness, especially in scenarios with a limited number of training instances.
- Active data collection:** Use TabPFN uncertainty to pick the next instance/seed, cutting measurement cost.